PROJECT REPORT

ON

LEGALEASE

AI-POWERED LEGAL DOCUMENT GENERATOR

A Full Project Documentation Report

Submitted in partial fulfillment of the requirements for the project

TEAM ID : SWTID-2026-1294

TEAM SIZE : 4

TEAM LEADER : Anitha M

TEAM MEMBER :Akash P

TEAM MEMBER : Murugadass R

TEAM MEMBER : Sakthivinayagan M

ACKNOWLEDGEMENT

I express my sincere gratitude to everyone who supported the development and documentation of the LegalEase project. The project combines generative AI, web application development, document formatting, and user-oriented design. I also acknowledge the value of the supplied project specification and sample output screens in documenting the application workflow.

ABSTRACT

LegalEase is an AI-powered legal document generator designed to simplify the preparation of structured legal documents. The application accepts inputs such as document type, parties involved, terms and conditions, and effective date, and uses a generative AI model to produce structured legal content.

The project architecture combines a Streamlit frontend, a FastAPI backend, a Gemini-based AI generation layer, and formatting utilities for TXT, DOCX, PDF, and HTML preview output. Users can review generated content, edit it, and download the result in multiple formats. The supplied sample screens demonstrate the empty form, filled form, generated preview, editing interface, DOCX output, and final DOCX page.

This report presents the complete project structure, functional workflow, system architecture, implementation milestones, user interface, export process, testing approach, and future enhancement areas described in the source material.

TABLE OF CONTENTS

1. Certificate
2. Declaration
3. Acknowledgement
4. Abstract
5. List of Figures
6. List of Tables
7. 1. Introduction
8. 2. Project Description
9. 3. Problem Statement and Proposed Solution
10. 4. Objectives
11. 5. Scope of the Project
12. 6. Use-Case Scenarios
13. 7. Requirements and Technology Stack
14. 8. System Architecture
15. 9. Project Workflow and Milestones
16. 10. Functional Modules
17. 11. Backend and API Design
18. 12. Frontend and User Interface
19. 13. Document Generation and Formatting
20. 14. Database and Data Handling Considerations
21. 15. Testing and Validation
22. 16. Deployment
23. 17. Screens and Demonstration
24. 18. Advantages and Limitations
25. 19. Future Enhancements
26. 20. Conclusion
27. Appendix A – Sample Inputs
28. Appendix B – Project Structure
29. Appendix C – Source and Screen Notes

LIST OF FIGURES

30. Figure 1 – High-level LegalEase system architecture
31. Figure 2 – Empty LegalEase form
32. Figure 3 – Filled LegalEase form
33. Figure 4 – Generated document: dark HTML preview
34. Figure 5 – Editing the generated document
35. Figure 6 – Downloaded DOCX: logo, terms table and footer
36. Figure 7 – DOCX last page: signature blocks and footer

LIST OF TABLES

37. Table 1 – Technology stack
38. Table 2 – Functional modules
39. Table 3 – API request fields
40. Table 4 – Testing checklist
41. Table 5 – Example user inputs

1. INTRODUCTION

1.1 Background

Legal documents commonly require structured information, consistent terminology, clauses, dates, parties, and professional formatting. LegalEase is presented as a system that uses generative AI to assist with this document-creation workflow.

1.2 Need for the Project

The supplied project material identifies users such as entrepreneurs, professionals, freelancers, landlords, and individuals who may need legal-document drafts without a legal background. LegalEase provides a guided workflow in which the user supplies the essential information and receives a structured draft.

1.3 Project Overview

The application is built around a Streamlit interface, a FastAPI backend, a Gemini AI integration, and document-formatting utilities. The source specifies generation, editable preview, and export to TXT, DOCX, and PDF.

1.4 Report Organization

This report progresses from project background and objectives to architecture, implementation milestones, functional modules, interface screens, testing, deployment, and future enhancement areas.

2. PROJECT DESCRIPTION

LegalEase leverages generative AI to create customizable and editable legal-document drafts. The source material identifies employment contracts, lease agreements, NDAs, and other agreements as examples.

2.1 Key Features

- AI-assisted legal-document generation
- Input-driven document customization
- Editable text preview
- Styled HTML preview
- TXT export
- DOCX export with formatting and branding
- PDF export with branding and footer
- Terms table generation
- Logo support
- Local FastAPI + Streamlit deployment

2.2 Intended Users

- Startup founders and entrepreneurs
- Freelancers and service providers
- Landlords and tenants
- Professionals and individuals requiring structured document drafts

3. PROBLEM STATEMENT AND PROPOSED SOLUTION

3.1 Problem Statement

Preparing a structured legal document can involve repeated drafting, formatting, and revision. Users may know the business terms they want to include but may not know how to turn them into a consistently structured document.

3.2 Proposed Solution

LegalEase provides a form-based interface for collecting the document type, parties, terms, and effective date. The backend forwards the structured request to the AI generation component. The generated content is then displayed for review and editing and can be exported.

3.3 Process Flow

42. User input
43. FastAPI request validation
44. Gemini document generation
45. Generated-text response
46. HTML preview
47. Optional editing
48. TXT/DOCX/PDF export

4. OBJECTIVES

- Develop an AI-powered legal document generation application.
- Provide a simple interface for entering document-specific information.
- Generate structured content based on user-provided details.
- Provide an editable preview before export.
- Support TXT, DOCX, and PDF downloads.
- Provide document branding and formatting.
- Separate frontend, backend, AI, and formatting responsibilities.
- Test the end-to-end workflow from input to final document.

5. SCOPE OF THE PROJECT

5.1 Included Scope

- Document generation
- Editable preview
- HTML rendering
- TXT/DOCX/PDF export
- Logo and footer formatting
- Terms-table generation
- FastAPI API layer
- Streamlit user interface

5.2 Document Types Mentioned

- Agreement
- Contract
- NDA (Non-Disclosure Agreement)
- Lease Agreement
- Employment Offer Letter
- Freelance Work Contract

5.3 Scope Boundary

The supplied source presents LegalEase as a document-generation and accessibility project. It does not provide evidence for a production legal-practice certification or for replacing professional legal advice. Accordingly, this report describes the system as an AI-assisted document-generation application.

6. USE-CASE SCENARIOS

6.1 Startup Founder – Employment Contract

A startup founder enters details about a new hire and selects “Employment Contract”. The system generates an agreement covering roles, responsibilities, compensation, and confidentiality. The user can customize the wording, add a company logo, and download a branded PDF.

6.2 Freelancer – NDA

A freelancer specifies the parties, confidentiality scope, and effective date to generate a structured NDA.

6.3 Landlord – Residential Lease

A landlord enters the property address, tenant details, and lease terms to create a formatted lease document with editable sections and a DOCX download.

7. REQUIREMENTS AND TECHNOLOGY STACK

7.1 Prerequisites

- FastAPI framework knowledge
- Google Generative AI / Gemini familiarity
- Streamlit frontend knowledge
- Python programming proficiency
- Python 3.10+ and pip

7.2 Technology Stack

Technology	Layer	Purpose
Python	Programming	Application implementation
Streamlit	Frontend	Inputs, preview, editing, downloads
FastAPI	Backend	API requests and routing
Gemini 1.5 Pro	AI Core	Generative document content
python-docx	Document formatting	DOCX generation
FPDF	Document formatting	PDF generation
Pillow	Image handling	Supporting image processing
requests	HTTP communication	Request handling
python-dotenv	Configuration	Environment/API-key management

7.3 Dependencies

The source lists: fastapi, uvicorn, streamlit, python-docx, fpdf, Pillow, requests, google-generativeai, and python-dotenv.

8. SYSTEM ARCHITECTURE

The project architecture separates the application into frontend, backend, AI integration, and formatting components.

STREAMLIT FRONTEND
User input • Preview • Editing • Downloads

↓

FASTAPI BACKEND
POST /generate • Pydantic validation • Routing

↓

GEMINI AI CORE
GeminiDocumentGenerator → Generated legal content
↓
Formatting: HTML Preview • TXT • DOCX • PDF

8.1 Frontend

The frontend collects document type, parties, terms, and dates. It also provides generation, editing, preview, and download controls.

8.2 Backend

FastAPI exposes the API layer. The source specifies a root endpoint and a POST /generate endpoint.

8.3 AI Integration

GeminiDocumentGenerator forms a structured prompt and sends it to Gemini through generate_content, after which the generated text is returned to the application.

8.4 Formatting Layer

Formatting functions convert generated text into HTML preview, DOCX, PDF, and TXT outputs.

9. PROJECT WORKFLOW AND MILESTONES

Milestone 1 – Model Selection and Architecture

Research and select the generative AI model, define the Streamlit/FastAPI/Gemini architecture, and prepare the environment.

Milestone 2 – Core Functionalities Development

Implement document generation, formatting, editable preview, and download functions.

Milestone 3 – API Logic Integration

Configure main.py, routes.py, Pydantic validation, and the POST /generate endpoint.

Milestone 4 – Frontend Development

Create the Streamlit interface, input fields, preview, editing workflow, and export controls.

Milestone 5 – Deployment

Prepare dependencies, environment variables, local backend/frontend launch, and end-to-end verification.

10. FUNCTIONAL MODULES

Module	Main Function	Output
Input Module	Collect document type, parties, terms and date	Structured request
AI Generation Module	Generate document content	Legal-document text
Preview Module	Render generated content	HTML preview
Editing Module	Allow user modification	Edited document text
DOCX Module	Create formatted Word document	DOCX file
PDF Module	Create branded PDF	PDF file
TXT Module	Create plain-text version	TXT file
API Module	Validate and route requests	API response

10.1 Core Utility Functions

- sanitize_text(text)
- format_docx(text, doc_type)
- format_pdf(text, doc_type)
- format_html_preview(text)

11. BACKEND AND API DESIGN

11.1 main.py

- Initialize FastAPI app
- Define / root endpoint
- Include routes.py router

11.2 routes.py

- Define Pydantic DocumentRequest
- Initialize GeminiDocumentGenerator
- Create POST /generate
- Return generated legal content

11.3 API Request Fields

Field	Description
document_type	Type of legal document
parties	People or organizations involved
terms	Terms and conditions; semicolon-separated clauses
dates	Effective date

12. FRONTEND AND USER INTERFACE

12.1 Page Layout

The source describes a Streamlit layout with a centered logo/title area and input controls. The supplied sample-output PDF provides six screens documenting the user interface and output sequence.

12.2 Input Controls

- Document Type
- Parties Involved
- Terms & Conditions
- Effective Date
- Generate Document
- Edit Document
- Download TXT / DOCX / PDF

12.3 Preview

The generated result is presented as a dark-themed, scrollable HTML preview. The user can review the generated text before editing or exporting.

13. DOCUMENT GENERATION AND FORMATTING

13.1 TXT

TXT provides the generated content as plain text.

13.2 DOCX

The source states that DOCX formatting can include a logo, Times New Roman font, headings, a terms table, and a footer.

13.3 PDF

The source describes PDF output with a centered logo, bold section headings, bullet-style terms, and header/footer formatting.

13.4 HTML Preview

The HTML preview converts generated text into styled HTML blocks for display within Streamlit.

14. DATA HANDLING AND SECURITY CONSIDERATIONS

The project description states that LegalEase is intended to provide secure handling of sensitive user data. The development workflow specifies using a .env file to store the Gemini API key rather than placing the key directly in application code.

14.1 Configuration

- Use environment variables for API credentials.
- Keep API keys outside source code.
- Validate incoming request fields using Pydantic.
- Treat generated legal text as user-provided sensitive content where applicable.

The supplied source does not provide a detailed database schema or a complete production security architecture; therefore this report does not invent one.

15. TESTING AND VALIDATION

15.1 Functional Test Cases

Test ID	Test Input/Action	Expected Result	Status
TC01	Open application	Empty form is displayed	To be verified
TC02	Enter document type, parties, terms and date	Values are accepted	To be verified
TC03	Click Generate Document	Generated content appears	To be verified
TC04	Open Edit Document	Editable content is displayed	To be verified
TC05	Download TXT	TXT file is produced	To be verified
TC06	Download DOCX	Formatted DOCX is produced	To be verified
TC07	Download PDF	Branded PDF is produced	To be verified
TC08	Check logo/footer	Branding appears in supported outputs	To be verified

The status column is intentionally marked “To be verified” because the supplied screen document identifies several screens as illustrative mockups and instructs final submission users to replace them with screenshots from a running application.

16. DEPLOYMENT

16.1 Local Deployment Steps

49. Create and activate a virtual environment.
50. Install dependencies from requirements.txt.
51. Create .env and configure the Gemini API key.
52. Launch FastAPI with Uvicorn.
53. Run the Streamlit frontend.
54. Perform end-to-end functional testing.

16.2 Deployment Options Mentioned in Source

The supplied project material mentions Render, Railway, Deta, Streamlit Community Cloud, Fly.io, and VPS-based deployment as possible options.

17. SCREENS AND DEMONSTRATION

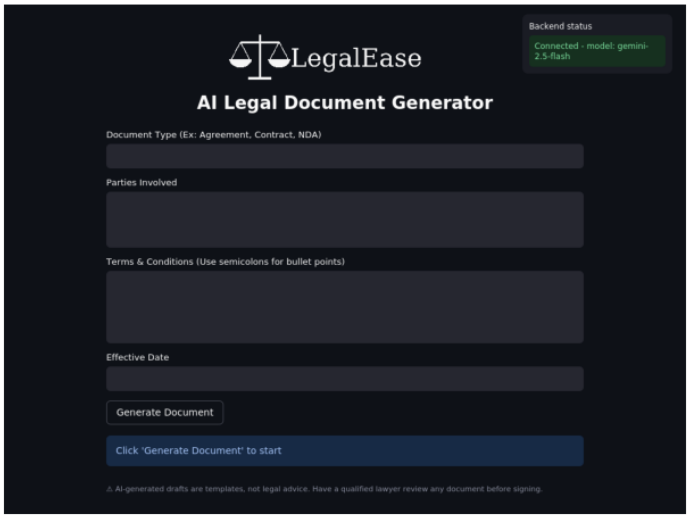
The following figures are taken from the uploaded “LegalEase_Sample_Output_Screens.pdf”. The source explicitly notes that Screens 1–4 are illustrative mockups of the Streamlit interface, while the DOCX pages and TXT output are produced by the project's real export code from a sample contract. The source also recommends replacing the mockups with screenshots from the running application for final submission. □filecite□turn1file0□L2-L7□

17.1 Screen 1 – Empty Form

Exploring LegalEase - Sample Output Screens

Note: Screens 1-4 are illustrative mockups of the Streamlit interface (drawn in HTML, using the project's real preview formatter). The DOCX pages and TXT output are produced by the project's real export code from a sample contract. Replace these with real screenshots from your running app (streamlit run frontend/app.py) for final submission.

1. Empty form



2. Filled form (example input: Freelance Work Contract)

Figure 2. Empty LegalEase input form.

17.2 Screen 2 – Filled Form

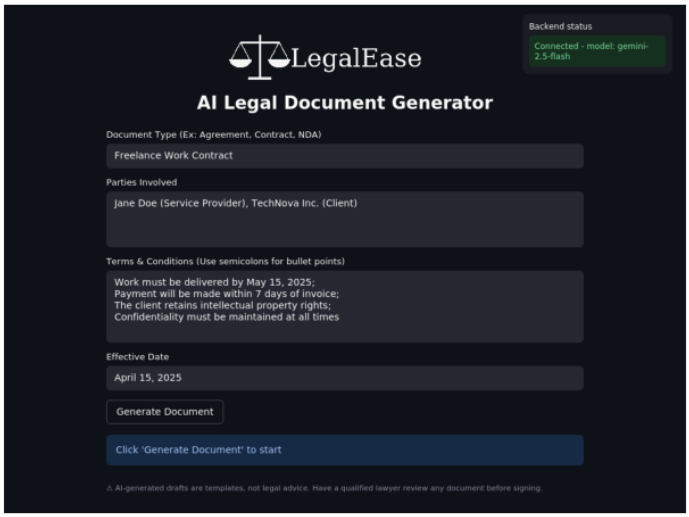


Figure 3. Filled LegalEase form using a Freelance Work Contract example.

17.3 Screen 3 – Generated Document

3. Document generated - dark HTML preview and download buttons

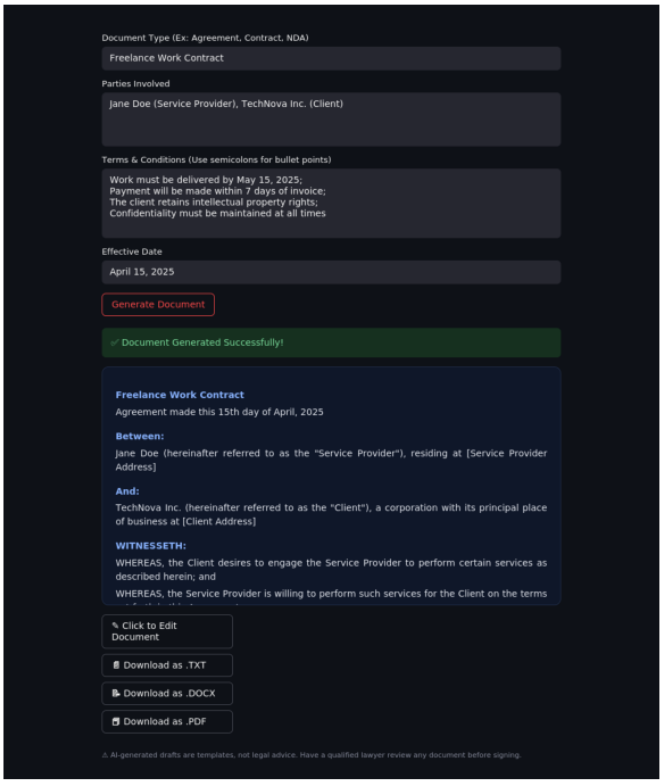


Figure 4. Generated document with dark HTML preview and download buttons.

17.4 Screen 4 – Editing

4. Editing the document

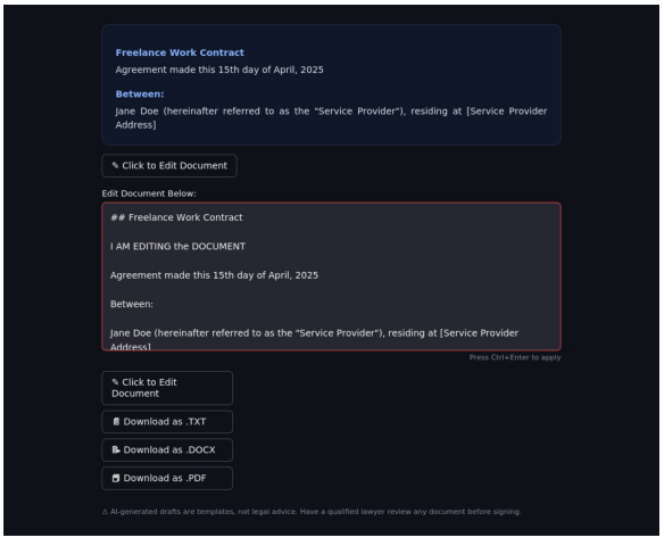
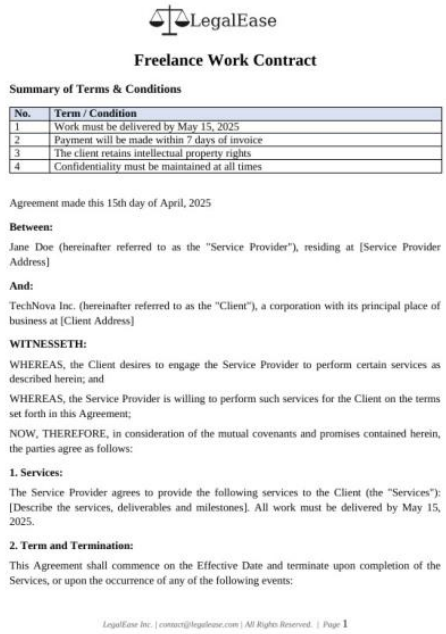


Figure 5. Editing the generated document.

17.5 Screen 5 – Downloaded DOCX

5. Downloaded .DOCX (logo, terms table, footer with page number)



LegalEase

Freelance Work Contract

Summary of Terms & Conditions

No.	Term / Condition
1	Work must be delivered by May 15, 2025
2	Payment will be made within 7 days of invoice
3	The client retains intellectual property rights
4	Confidentiality must be maintained at all times

Agreement made this 15th day of April, 2025

Between:

Jane Doe (hereinafter referred to as the "Service Provider"), residing at [Service Provider Address]

And:

TechNova Inc. (hereinafter referred to as the "Client"), a corporation with its principal place of business at [Client Address]

WITNESSETH:

WHEREAS, the Client desires to engage the Service Provider to perform certain services as described herein; and

WHEREAS, the Service Provider is willing to perform such services for the Client on the terms set forth in this Agreement;

NOW, THEREFORE, in consideration of the mutual covenants and promises contained herein, the parties agree as follows:

1. Services:

The Service Provider agrees to provide the following services to the Client (the "Services"): [Describe the services, deliverables and milestones]. All work must be delivered by May 15, 2025.

2. Term and Termination:

This Agreement shall commence on the Effective Date and terminate upon completion of the Services, or upon the occurrence of any of the following events:

LegalEase Inc. | contact@legalease.com | All Rights Reserved. | Page 1

Figure 6. Downloaded DOCX showing logo, terms table and footer with page number.

17.6 Screen 6 – DOCX Last Page



Figure 7. DOCX last page showing signature blocks and footer.

18. ADVANTAGES AND LIMITATIONS

18.1 Advantages

- Guided user input
- AI-assisted generation
- Editable output
- Multiple download formats
- Branding and formatting support
- Clear separation of application layers
- Suitable for local deployment and future expansion

18.2 Limitations Identified from the Source

- Screens 1–4 in the uploaded screen document are illustrative mockups.
- Final submission should use screenshots from the running application.
- The supplied material does not provide a full database implementation.
- The supplied material does not establish that generated documents constitute professional legal advice.

19. FUTURE ENHANCEMENTS

The supplied conclusion identifies future directions including deeper contract analysis, integration with legal databases, personalized recommendations, multilingual capabilities, and continued refinement through testing and feedback.

- Deeper contract analysis
- Legal database integration
- Personalized recommendations
- Expanded multilingual support
- More document templates
- Additional validation and testing
- Cloud deployment

20. CONCLUSION

LegalEase demonstrates an end-to-end AI-assisted workflow for generating structured legal-document drafts. The project brings together Streamlit, FastAPI, Gemini, and document-formatting utilities to collect user inputs, generate content, preview and edit it, and export the result.

The supplied screens document the intended interface and output sequence: an empty form, a filled form, generated preview, editing, DOCX output, and the final DOCX page. The screen document also clearly distinguishes illustrative UI mockups from the sample DOCX output and recommends replacing mockups with live application screenshots for final submission. filecite turn1file0 L2-L15

Overall, the project provides a structured foundation for AI-assisted legal document drafting and presents a clear separation of concerns between user interface, API, AI generation, and document formatting.

APPENDIX A – SAMPLE INPUTS

Input	Example
Document Type	Freelance Work Contract
Parties	Jane Doe (Service Provider), TechNova Inc. (Client)
Terms	Payment within 30 days; delivery by agreed deadline; confidentiality; termination with 15 days notice
Effective Date	April 15, 2025

APPENDIX B – PROJECT STRUCTURE

- main.py – FastAPI application initialization and root/router configuration.
- routes.py – Pydantic request model and /generate endpoint.
- ai_core/gemini_generator.py – Gemini document generation logic.
- Streamlit frontend – user input, preview, editing and downloads.
- Formatting utilities – TXT, DOCX, PDF and HTML preview generation.
- .env – Gemini API-key configuration.
- requirements.txt – Python dependencies.

APPENDIX C – SOURCE AND SCREEN NOTES

Primary project source: the supplied LegalEase project material describing the project, prerequisites, architecture, milestones, functions, deployment workflow, use cases, and conclusion.

Primary screen source: the uploaded LegalEase_Sample_Output_Screens.pdf. Its first four screens are identified by the source as illustrative Streamlit mockups; the DOCX pages and TXT output are described as being produced by the project's real export code from a sample contract. For final academic submission, the source recommends capturing the application while running and replacing the mockups with live screenshots. filecite turn1file0 L2-L7

This report deliberately avoids inventing unsupported database structures, deployment results, or production claims.